

Name $\rightarrow$ Komal Choudhury

DSA

- 1) Ans:- (b) Data and a pointer to the next code
- 2) Ans:- (c) NULL
- 3) Ans:- (b) Self-referential structure
- 4) Ans:- (b) head == NULL
- 5) Ans:- (b) newnode $\rightarrow$ next = head; head = newnode;
- 6) Ans:- (b) temp = head; head = head $\rightarrow$ next; free (-temp);
- 7) Ans:- (c) Traversing from tail to head
- 8) Ans:- (b) n
- 9) Ans:- (c) Copying the data of P $\rightarrow$ next into P, then deleting P $\rightarrow$ next
- 10) Ans:- (b) while (P $\neq$ NULL)
- 11) Ans:- (b) P $\rightarrow$ next == NULL
- 12) Ans:- (c) Two pointers advancing at different
- 13) Ans:- (c) Three
- 14) Ans:- (b) Exactly and node
- 15) Ans:- (b) P = (struct node *) malloc (size of (struct node))
- 16) Ans:- (b) void special-case handling for insertion and deletion at the head
- 17) Ans:- (c) The first node
- 18) Ans:- (b) It does not support random (indexed) access
- 19) Ans:- (b) Low of access to all nodes, causing a memory leak.
- 20) Ans:- (b) A slow pointer and a fast pointer
- 21) Ans:- (c) ~~Recursion~~ The last node
- 22) Ans:- (b) Nodes may be scattered anywhere in memory

23) Ans:- struct mode {

```
    int data;  
    struct mode * next;  
};
```

24) Ans:- The code prints the sum of each node and its next node:

→ 10 + 20 = 30

→ 20 + 30 = 50

→ 30 + 40 = 70

Output is → 30 50 70

25) Ans:- Error:- head is passing by value, so changes to head pointer won't affect the original list. Also, an empty list case needs handling

correct version:-

```
void insertEnd(struct mode * head, int n)
```

```
{  
    struct mode * t = (struct mode*) malloc  
        (size of (struct mode));
```

```
    t->data = n;
```

```
    t->next = NULL;
```

```
    if (*head == NULL) {
```

```
        *head = t;
```

```
}
```


26) Ans:- `struct node * reverse ( struct node * head ) {`

`struct node * prev = NULL;`

`struct node * curr = head;`

`struct node * next = NULL;`

`while ( curr != NULL ) {`

`next = curr -> next;`

`curr -> next = prev;`

`prev = curr;`

`curr = next;`

`{`
`return prev;`

`}`

27) Ans:- Suppose, the list is `10 -> 20 -> 30 -> 40 -> NULL`

=> I want to delete 30.

=> Then, first 20 node's next pointer.

needs to be pointed to 40.

Before `-> 10 -> 20 -> 30 -> 40 -> NULL`

prev del next

After `-> 10 -> 20 - - - - -> 40 - - -> NULL`

↑
30 deleted

`prev -> next = del -> next`

`free (del);`

28) Initial list :-

$5 \rightarrow 8 \rightarrow 12 \rightarrow 3 \rightarrow \text{NULL}$

(i) $7 \rightarrow 5 \rightarrow 8 \rightarrow 12 \rightarrow 3 \rightarrow \text{NULL}$

(ii) $7 \rightarrow 5 \rightarrow 8 \rightarrow 3 \rightarrow \text{NULL}$

(iii) $7 \rightarrow 5 \rightarrow 8 \rightarrow 3 \rightarrow 2 \rightarrow \text{NULL}$

Final answer :- $7 \rightarrow 5 \rightarrow 8 \rightarrow 3 \rightarrow 9 \rightarrow \text{NULL}$